



COURSEWORK SPECIFICATION

COMM040 - Text Mining and Natural Language Processing

Module Leader: Rodrigo Wilkens

Academic Year: 2025/26

Information Retrieval

Implement a lexical and dense IR ranking system and evaluate retrieval using Precision@k and MAP using the cranfield dataset

Environment Setup

1. API
 - `pip install ir_datasets`
 - `pip install rank-bm25`
 - `pip install ir-measures`
 - `pip install sentence-transformers`
 2. Dataset
 - `import ir_datasets`
 - `dataset = ir_datasets.load("cranfield")`
-

Dataset Structure

1. From the dataset, extract:
 - a. Documents (`doc_id`, `text`)
 - b. Queries (`query_id`, `text`)
 - c. Qrels (`query_id`, `doc_id`, `relevance`)
-

Build a TF-IDF Retrieval System

1. Required API
 - a. `from sklearn.feature_extraction.text import TfidfVectorizer`
 - b. `from sklearn.metrics.pairwise import cosine_similarity`
 2. Vectorise all Cranfield documents using TF-IDF.
 3. Encode one query.
 4. Compute cosine similarity between query and all documents.
 5. Rank documents by similarity score.
 6. Return the top-10 documents.
-

BM25 Retrieval

1. Required API

- a. `from rank_bm25 import BM25Okapi`
 2. Build a BM25 index from all documents.
 3. Score the same query.
 4. Retrieve top-10 documents.
 5. Compare ranking with TF-IDF.
-

Evaluation with Cranfield Qrels (for at least 5 queries)

1. Retrieve top-10 documents using TF-IDF.
 2. Compute evaluation metrics using the official qrels.
 3. Repeat with BM25.
 4. Compare average performance.
-

Dense Retrieval

1. Required API
 - a. `from sentence_transformers import SentenceTransformer`
 - b. `model = "all-MiniLM-L6-v2"`
 2. Encode all documents into dense embeddings.
 3. Encode query embeddings.
 4. Compute cosine similarity.
 5. Rank documents.
 6. Compare results with lexical retrieval.
 7. Evaluate using MAP.
-